COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 / Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | September 1, 2026 | **DATE SUBMITTED:** | September 1, 2026 |
| **NAME:** | Reyes, Fenzie Aaron A. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## SEATWORK NO. 7.2 ALGORITHM COMPLEXITY

### 1. PROCEDURE OUTPUT

What procedures did you perform?

**1. Finding an Element**

```
for (int i = 0; i < n; i++)   // O(n)
{
    if (arr[i] == target)   // O(1) per iteration
    {
        return i;   // O(1)
    }
}
```

**Best-Case Time Complexity: O(1)**
**Worst-Case Time Complexity: O(n)**
**Brief Explanation:** If the target is the first element, the program finds it right away, so the best case for this one is O(1). If the target is at the end or not in the array, it has to check everything, making the O(n) the worst case.

**2. Printing All Elements**

```
for (int i = 0; i < n; i++)   // O(n)
{
    cout << arr[i] << endl;   // O(1) per iteration
}
```

**Best-Case Time Complexity: O(n)**
**Worst-Case Time Complexity: O(n)**
**Brief Explanation:** The program has to go through every element to print them all. Since it always does this, both the best and worst cases are O(n).

### 3. Nested Loops

```
for (int i = 0; i < n; i++)   // O(n)
{
    for (int j = 0; j < n; j++)   // O(n) for each i
    {
        cout << i << " " << j << endl;   //O(1) per iteration
    }
}
```

**Best-Case Time Complexity: $O(n^2)$**
**Worst-Case Time Complexity: $O(n^2)$**
**Brief Explanation:** The first loop runs n times, and for every one of those, the second loop also runs n times. This gives n x n = $O(n^2)$.

### 4. Conditional Processing

```
if (n > 100)   // O(1)
{
    for (int i = 0; i < n; i++)   // O(n)
    {
        cout << arr[i];   // O(1) per iteration
    }
}
else
{
    cout << "Small input";   //O(1)
}
```

**Best-Case Time Complexity: O(1)**
**Worst-Case Time Complexity: O(n)**
**Brief Explanation:** If n is 100 or less, the program only prints a message, so it takes O(1). If n is greater than 100, it goes through all the elements, giving O(n).

### 5. Fixed Inner Loop

```
for (int i = 0; i < n; i++)   // O(n)
{
    for (int j = 0; j < 10; j++)   // O(10) = O(1)
    {
        cout << i << j;   // O(1)
    }
}
```

**Best-Case Time Complexity: O(n)**
**Worst-Case Time Complexity: O(n)**
**Brief Explanation:** The inner loop only runs 10 times, which is a fixed number and doesn't grow with n. Since the outer loop runs n times, the overall complexity is O(n).

**6. Two Consecutive Loops**

```
for (int i = 0; i < n; i++)   // O(n)
{
    cout << arr[i];   // O(1) per iteration
}

for (int j = 0; j < n; j++)   // O(n)
{
    cout << arr[j] * 2;   O(1) per iteration
}
```

**Best-Case Time Complexity: O(n)**
**Worst-Case Time Complexity: O(n)**
**O(n) + O(n) = O(2n) = O(n)**
**Brief Explanation:** Both loops go through n elements, so together they do about 2n operations. We ignore the constant 2, leaving O(n).

**7. Dividing by Two**

```
int x = n;   // O(1)

while (x > 1)   // O(log n)
{
    x = x / 2;   // O(1) per iteration
}
```

**Best-Case Time Complexity: O(1)**
**Worst-Case Time Complexity: O(log n)**
**Brief Explanation:** Every time the loop runs, x is cut in half, so it reaches 1 pretty quickly. This gives a worst-case complexity of O(log n), while the best case is O(1).

**8. Nested Loop With Different Limits**

```
for (int i = 0; i < n; i++)   // O(n)
{
    for (int j = 0; j < i; j++)   // O(i)
    {
        cout << i << j;   // O(1)
    }
}
```

**Best-Case Time Complexity: $O(n^2)$**
**Worst-Case Time Complexity: $O(n^2)$**

- **The inner loop depends on i that is why it gives: 0 + 1 + 2 + … + (n-1) = $O(n^2)$**

**Brief Explanation:** The inner loop runs more times as "i" gets larger, creating a total that grows roughly like $n^2$. Therefore, both cases are $O(n^2)$.

**9. Early Termination**

```
for (int i = 0; i < n; i++)   // O(n)
{
    if (arr[i] == target)   // O(1) per iteration
    {
        cout << "Found";   // O(1)
        break;    // O(1)
    }
}
```

**Best-Case Time Complexity: O(1)**
**Worst-Case Time Complexity: O(n)**
**Brief Explanation:** If the target is found immediately, the program stops after one check, so the best case is O(1). If it has to check the whole array, the worst case is O(n).

**10. Two-Level Search**

```
for (int i = 0; i < n; i++)   // O(n)
{
    for (int j = 0; j < n; j++)   // O(n) for each i
    {
        if (arr[i] == target)   // O(1)
        {
            break;   // O(1)
        }
    }
}
```

**Best-Case Time Complexity: O(n)**
**Worst-Case Time Complexity: $O(n^2)$**
**Brief Explanation:** In the best case, the inner loop stops quickly, but the outer loop can still go through all n elements, giving O(n). If the condition never causes an early stop, both loops run fully, resulting in $O(n^2)$.

**11. Two-Level Search**

```
for (int i = 0; i < n; i++)   // O(n)
{
    int x = n;   // O(1)

    while (x > 1)   // O(log n)
    {
        x = x / 2;   // O(1) per iteration
    }
}
```

**Best-Case Time Complexity: O(n log n)**
**Worst-Case Time Complexity: O(n log n)**
**Brief Explanation:** The outer loop runs n times, and each time it contains a loop that takes O(log n). Since these are nested, we multiply them to get O(n log n).

### 12. Conditional Nested Loop

```
if (arr[0] == target)   // O(1)
{
    cout << "Found";   // O(1)
}
else
{
    for (int i = 0; i < n; i++)   // O(n)
    {
        for (int j = 0; j < n; j++)   // O(n)
        {
            cout << arr[i] + arr[j];    // O(1)
        }
    }
}
```

**Best-Case Time Complexity: O(1)**
**Worst-Case Time Complexity: O($n^2$)**
**Brief Explanation:** If the first element matches the target, the program finishes almost immediately, so the best case is O(1). Otherwise, the two loops run through all combinations, making the worst case O($n^2$).

### 13. Two-Level Search

```
int max = arr[0];   // O(1)

for (int i = 1; i < n; i++)   // O(n)
{
    if (arr[i] > max)   // O(1) per iteration
    {
        max = arr[i];   // O(1)
    }
}
```

**Best-Case Time Complexity: O(n)**
**Worst-Case Time Complexity: O(n)**
**Brief Explanation:** The program needs to check every element to make sure it has found the largest one. Because of this, both the best and worst cases are O(n).

**14. Three Nested Loops**

```
for (int i = 0; i < n; i++)   // O(n)
{
    for (int j = 0; j < n; j++)   // O(n)
    {
        for (int k = 0; k < n; k++)   // O(n)
        {
            cout << i + j + k;   // O(1)
        }
    }
}
```

**Best-Case Time Complexity: $O(n^3)$**
**Worst-Case Time Complexity: $O(n^3)$**
**Brief Explanation:** Each of the three loops runs n times, so their work is multiplied: n x n x n = $n^3$. Therefore, the complexity is $O(n^3)$.

**15. Comprehensive Analysis**

```
for (int i = 0; i < n; i++)   // O(n)
{
    if (arr[i] == target)   // O(1) per iteration
    {
        return i;   // O(1)
    }

    int x = n;   // O(1)

    while (x > 1)   // O(log n)
    {
        x = x / 2;   // O(1) per iteration
    }
}
```

**Best-Case Time Complexity: O(1)**
**Worst-Case Time Complexity: O(n log n)**
**Brief Explanation:** If the target is found on the first check, the program stops immediately, giving a best case of O(1). If the target is not found, the n iterations each perform O(log n) work, resulting in O(n log n).

COMPUTER ENGINEERING

QUEZON CITY

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** Grammarly

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** For grammar correction.

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** I only checked if my grammar was correct. Nothing more than that was used to answer or replace my original idea with those answers.

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.